

C++

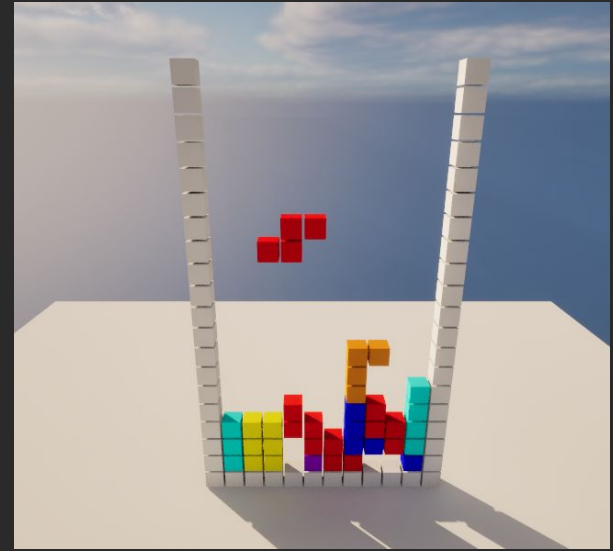
Vojtěch Cimbura

Lecture I

21.10.2025

Outline

- We will create a simplified Tetris using C++ in UE
- The plan:
 - 1) Create a visual representation of a block
 - 2) React to the player's input
 - 3) Implement the game logic
- To start with, Create a new UE C++ project
 - Open UE
 - Games tab, Blank project
 - Specify location, project name e.g. "Tetris"
 - Project Defaults - C++ project
- Let UE generate it, both VS and UE editor will open.



Introduction

- UE uses C++
- Blueprints are just a cover over C++ classes, that make fast prototyping possible (next slide)
- You can avoid C++, but having a balance between C++ and Blueprints is a key to make game development easier and create a “good game”
 - Finding that balance is not easy, one has to experiment and get experience using both Blueprints and C++

UNREAL
ENGINE

Introduction



```
UCLASS(MinimalAPI)
class UK2Node_MakeArray : public UK2Node_MakeContainer
{
    GENERATED_UCLASS_BODY()

public:
    // UEdGraphNode interface
    virtual FText GetNodeTitle(ENodeTitleType::Type TitleType) const override;
    virtual FText GetTooltipText() const override;
    virtual FSlateIcon GetIconAndTint(FLinearColor& OutColor) const override;
    virtual void ValidateNodeDuringCompilation(class FCompilerResultsLog& MessageLog) const override;
    // End of UEdGraphNode interface

    // UK2Node interface
    virtual void GetNodeContextMenuActions(class UToolMenu* Menu, class UGraphNodeContextMenuContext* Context) const override;
    virtual bool IncludeParentNodeContextMenu() const override { return true; }
    virtual class FNodeHandlingFunctor* CreateNodeHandler(class FKismetCompilerContext& CompilerContext) const override;
    virtual FText GetMenuCategory() const override;
    // End of UK2Node interface

    // UK2Node_MakeContainer interface
    virtual FName GetOutputPinName() const override;
    // UK2Node_MakeContainer interface
};
```

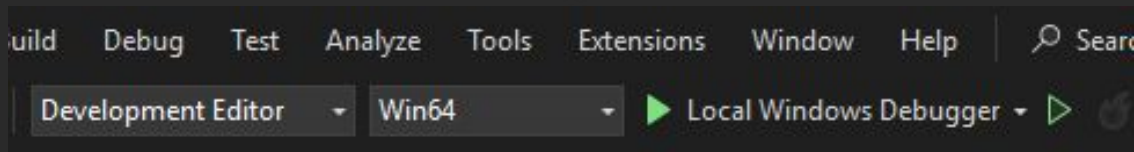
C++ Course?

- You should be familiar with basic programming in C++ and OOP paradigm
- We will make an introduction into C++ in UE, which has some specific differences from C++ that you are used to
 - `std::min()` → `FMath::Min()`
- We are NOT going to dive into generics or optimizations
 - The aim is to provide a head start for your future development in UE

UNREAL
ENGINE

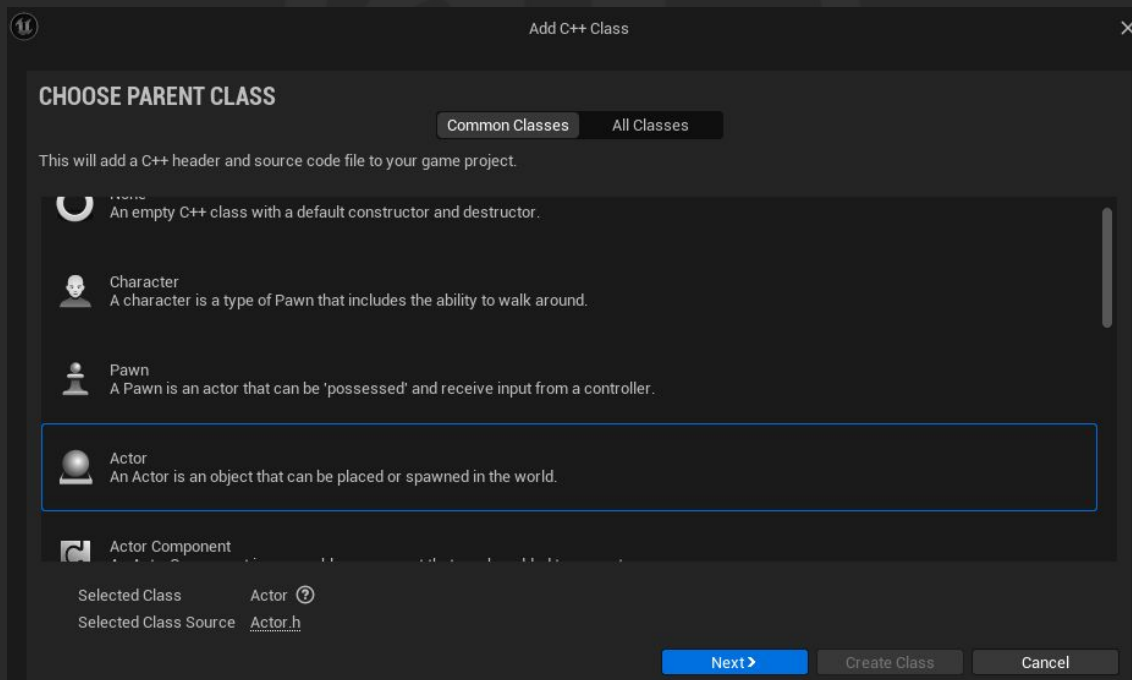
Opening the project

- Multiple ways:
 - Using the Epic Games Launcher
 - New: Using Visual Studio
 - With attached Debugger
 - Without debugger
- Multiple configurations and platforms
- Use based on your current needs - developing vs debugging



Basics - Creating a C++ class

- Let's create our first C++ class - **TetrisBlock**
- Creating a C++ class: Tools → New C++ Class...



Basics - Creating a C++ class

- Specify the name, you can leave Class Type and Path on default
- Notice that both **.h** and **.cpp** files will be created

NAME YOUR NEW ACTOR

Enter a name for your new class. Class names may only contain alphanumeric characters, and may not contain a space.
When you click the "Create" button below, a header (.h) file and a source (.cpp) file will be made using this name.

Class Type: **Public** Private

Name:

Path:

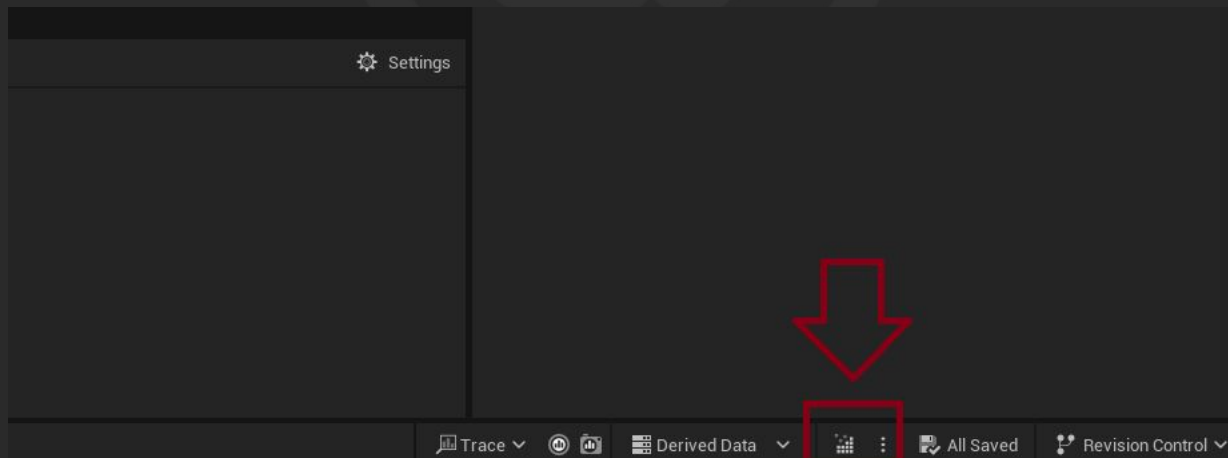
Header File:

Source File:

<Back Create Class Cancel

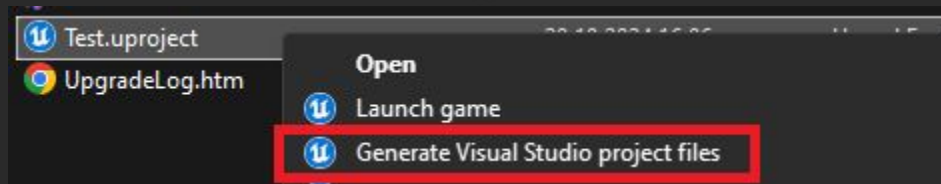
Hot reload

- Compile code changes while the editor is running
- = Patch the editor with code changes
- Be careful with this feature
 - Changes to properties exposed to blueprints might cause BP graph issues



Regenerate project

- At this point you might see errors in VS while viewing our newly added C++ class in the solution
- Close everything and RMB click on the **.uproject** file → Generate Visual Studio project files
 - This will refresh the solution with new build dependencies - new files
 - Think of it as creating a new file in your project and running CMake configure command



```

1 // Fill out your copyright notice in the Description page of Project Settings.
2
3 #pragma once
4
5 #include "CoreMinimal.h"
6 #include "GameFramework/Actor.h"
7 #include "TetrisBlock.generated.h"
8
9 UCLASS()
10 class TETRISOLUTION_API ATetrisBlock : public AActor
11 {
12     GENERATED_BODY()
13
14 public:
15     // Sets default values for this actor's properties
16     ATetrisBlock();
17
18 protected:
19     // Called when the game starts or when spawned
20     virtual void BeginPlay() override;
21
22 public:
23     // Called every frame
24     virtual void Tick(float DeltaTime) override;
25
26 };
27

```

```

1 // Fill out your copyright notice in the Description page of Project Settings.
2
3 #pragma once
4
5 #include "CoreMinimal.h"
6 #include "GameFramework/Actor.h"
7 #include "TetrisBlock.generated.h"
8
9 UCLASS()
10 class TETRISOLUTION_API ATetrisBlock : public AActor
11 {
12     GENERATED_BODY()
13
14 public:
15     // Sets default values for this actor's properties
16     ATetrisBlock();
17
18 protected:
19     // Called when the game starts or when spawned
20     virtual void BeginPlay() override;
21
22 public:
23     // Called every frame
24     virtual void Tick(float DeltaTime) override;
25
26 };
27

```

Naming convention in UE

```
AActor* Actor;
```

- First letter capitalized, no underscore between words
- Type names prefixed with an additional upper-case letter to distinguish them from variable names

Templated classes	T
Classes that inherit from UObject	U
Classes that inherit from AActor	A
Enums	E
Booleans	b (bool bIsActive = false;)
Structs	F

UObject

- Base class for objects in UE, the engine manages them
- No constructor arguments
 - All C++ UObjects are initialized on engine startup, and the engine calls their default constructor - **important to not write any game logic into constructor** otherwise the editor won't even open due to errors
 - You can write here subobject setup
 - Eg. Mesh initialization, Actor components init, Ticking properties
- Functionality provided by UObjects:
 - **Garbage collection**
 - **Reference updating**
 - **Reflection Serialization**
 - ...

```

1 // Fill out your copyright notice in the Description page of Project Settings.
2
3 #pragma once
4
5 #include "CoreMinimal.h"
6 #include "GameFramework/Actor.h"
7 #include "TetrisBlock.generated.h"
8
9 UCLASS()
10 class TEIRISSOLUTION_API ATetrisBlock : public AActor
11 {
12     GENERATED_BODY()
13
14 public:
15     // Sets default values for this actor's properties
16     ATetrisBlock();
17
18 protected:
19     // Called when the game starts or when spawned
20     virtual void BeginPlay() override;
21
22 public:
23     // Called every frame
24     virtual void Tick(float DeltaTime) override;
25
26 };
27

```

Code Reflection System

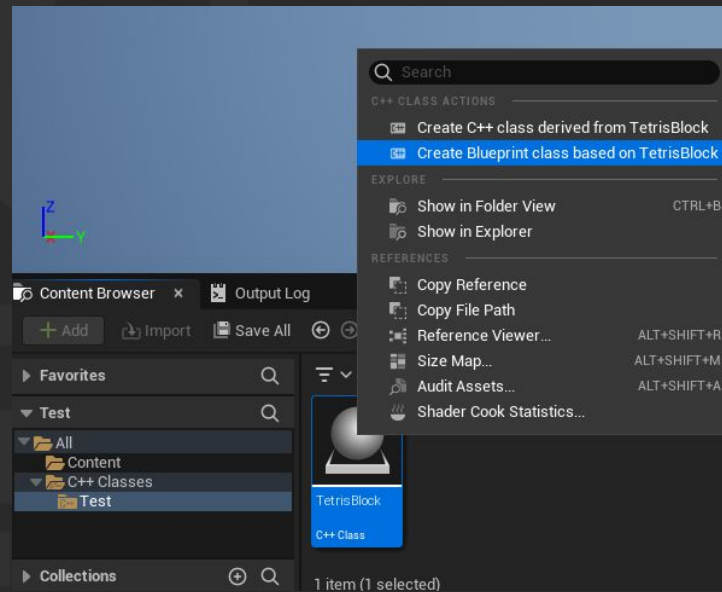
- Unreal Header Tool (UHT) uses macros to create code reflection in order for `UObjects` to function properly - Engine and Editor functionality
 - Registering variables, functions, classes:
 - `UPROPERTY`, `UFUNCTION`, `UCLASS`, `UENUM`
- There are parameters to above macros, that further specify their usage
 - [Unreal Engine Reflection System](#)
- Generated data stored in `<filename>.generated.h`
 - Always included as the **last** include in the header file
- Some UE capabilities require that your functions / variables are tagged with those macros
 - Such as timer functions, variables that you don't want to be GCed

TetrisBlock - Variables

- Right now we have a cookbook for an object that Engine knows about and can handle, but there is no visualisation or logic
- Let's add variables:
 - `UStaticMeshComponent* MeshComponent;`
 - `UMaterialInstance* BlockMaterialInstance;`
 - `UMaterialInstanceDynamic* DynamicBlockMaterialInstance;`
 - `FName ColorParameterName;`
 - `FColor BlockColor;`
- Don't forget to mark all as `UPROPERTY()`
 - `BlockMaterialInstance` and `ColorParameterName` name have to be `EditAnywhere` - will be assignable in BP from the Editor

TetrisBlock - Blueprint

- Now we want to create a Blueprint actor out of this C++ class in order to assign props in the Editor
 - Let's go to the Editor
- Create **BP_TetrisBlock** blueprint



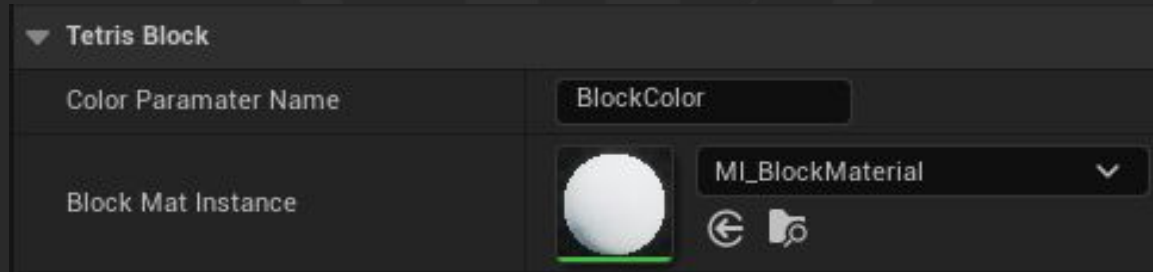
TetrisBlock - Material

- We need to specify a material for the block, which will let us change the color of the block by changing shader parameter in code
- Create a new material **M_BlockMaterial** with 'BlockColor' **VectorParameter** linked to **BaseColor**
- Create a material instance based on this material, name it **MI_BlockMaterial**



TetrisBlock - Blueprint

- Assign the created material instance `MI_BlockMaterial` in properties and set the color parameter to the same one we specified in the material



TetrisBlock - Functions

- We'd like to have a basic cube mesh on the actor, assign the dynamic material to the mesh and be able to read and assign a mesh color
- **Constructor**
 - Load a default cube mesh from the engine
 - Set the cube mesh as a Root Component of the actor
- **BeginPlay()**
 - Check if material is assigned
 - Create dynamic material instance and set the material to the mesh component
 - (Later) Set visibility of the actor to false
- Create Getter & Setter for **BlockColor** variable
 - Setter will also set material color - **SetVectorParameterValue()**
 - (Later) If the new color is black, hide the actor

Asserts in UE

- `check()` / `checkf()`

- Behaves like `assert` in C/C++
- Failed condition **stops execution**, shows dialog, breakpoint
- Does not evaluate the condition in optimized build

- `verify()` / `verifyf()`

- Similar to `check`, **stops execution**
- Makes sure that evaluated condition is not optimized out in optimized build configurations

- `ensure()` / `ensureMsgf()`

- Similar to `verify`, but **does not stop execution** -> non fatal errors
- Failure reported only once
 - use `ensureAlways()` if you want to report always

Next steps

- We have the board building block set up - player input and the game logic left
- Player input
 - Utilize Enhanced Input similarly as for the Arkanoid project
 - Setup input actions
 - Create `TetrisGameMode` and set up our custom Pawn
 - Print out debug messages into the Output Log to verify functionality
- Game Mode
 - Let's use it to put all game logic in there
 - Player's Pawn has access to the game mode:
 - Direct access - quick implementation, but messy solution
 - Delegates - slower implementation, but preferred way

Learning material

- [UE Naming convention](#)
- [Programming in UE C++, the whole documentation](#)
- [Balancing BPs and C++](#)
- [Getting started with Unreal Engine](#)
- [Asserts in UE](#)
- [Objects in UE](#)

UNREAL
ENGINE